

A Formal Mechanism for Relating Automata Ambiguity and JavaScript Backtracking Traces

ANONYMOUS AUTHOR(S)

Backtracking regular-expression engines expose a gap between two familiar views of regular expressions. Automata theory explains nondeterminism through accepting runs and their degree of ambiguity, while production engines explain matching through an ordered backtracking execution that may explore many failing and successful alternatives before returning the highest-priority match. The gap is especially visible in JavaScript, whose semantics is specified operationally and whose performance pathologies are central to Regular Expression Denial of Service (ReDoS).

This paper develops a mechanized bridge between these views. We use position automata as a common representation and relate accepting runs of a Glushkov-style automaton to leaves in a JavaScript-style backtracking trace tree. The resulting correspondence explains when the size and growth of backtracking search are controlled by classical ambiguity notions: unambiguous automata induce at most one successful trace, finite ambiguity yields a uniform bound on successful alternatives, and polynomial or exponential ambiguity gives a semantic classification of families of inputs that may drive ReDoS behavior. The development is being mechanized in Coq. The current artifact formalizes regular and positioned syntax, Kleene semantics, position-set constructions, executable automata, and basic ambiguity definitions; the remaining proof obligations are isolated as the trace/automaton correspondence and the Weber–Seidl ambiguity classification layer.

CCS Concepts: • **Theory of computation** → *Automata over infinite objects*; • **Software and its engineering** → **Formal methods**; • **Security and privacy** → *Logic and verification*.

Additional Key Words and Phrases: regular expressions, ambiguity, JavaScript, backtracking semantics, ReDoS, Coq

ACM Reference Format:

Anonymous Author(s). 2026. A Formal Mechanism for Relating Automata Ambiguity and JavaScript Backtracking Traces. 1, 1 (May 2026), 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern regular-expression engines are not merely membership testers for regular languages. A JavaScript engine must return the match selected by an ordered backtracking policy, including the effects of alternation priority, greedy and lazy quantifiers, captures, lookaround, and other language features. Recent work by Barriere, Deng, and Pit-Claudel gives a mechanized semantics for JavaScript regular expressions in which the result of matching is not just a single top-priority match, but a complete ordered backtracking tree recording the matches and failures considered by the engine [1]. That tree is an attractive semantic object for verification: it is faithful to backtracking while still being inductive and suitable for proof.

At the same time, the classical theory of finite automata has long studied the number of accepting runs of a nondeterministic finite automaton (NFA) on a word. This number is the word’s ambiguity, and the supremum over all words is the degree of ambiguity of the automaton. Weber and Seidl showed that ambiguity is not just a coarse counting measure: finite ambiguity admits structural bounds, infinite ambiguity has a simple polynomial-time criterion, and the polynomial degree of ambiguity growth is computable [2]. These results are tantalizing for ReDoS, because catastrophic backtracking is precisely a phenomenon in which a compact regular expression induces a very large search space on a family of inputs.

This paper asks whether these two accounts can be made to meet:

Can the growth of JavaScript-style backtracking traces be explained, bounded, and eventually certified through automata ambiguity?

Our answer is a formal mechanism that connects successful leaves of a backtracking trace to accepting runs of a position automaton. The intended scope is deliberately staged. We first treat the regular core of JavaScript regular expressions, where Glushkov position automata give a syntax-directed NFA whose states are occurrences of atoms. We then use the tree semantics of Barriere et al. as the target view of engine execution. The central theorem states that, after erasing priority annotations and failing leaves, successful trace leaves are in bijection with accepting marked runs of the position automaton. This theorem is the bridge that allows ambiguity results to be transported to backtracking traces.

The bridge gives three payoffs.

- It separates *semantic nondeterminism* from *engine priority*. Ambiguity counts the possible successful decompositions of a word; the backtracking policy orders those decompositions and selects the first.
- It explains why unambiguity and finite ambiguity are useful ReDoS invariants. If the constructed automaton is unambiguous or finitely ambiguous, then the successful part of the trace tree is correspondingly bounded.
- It identifies the next verification target: lift Weber–Seidl criteria into Coq as certificates for infinite or polynomial ambiguity growth, and use those certificates to justify static ReDoS warnings.

This is an early-stage ICFP draft. The Coq development already contains a substantial regular-expression and automata core, but the full bridge theorem and the Weber–Seidl classification proof are not complete. We state them as the main technical obligations of the paper rather than as finished results.

Contributions. This draft makes the following contributions.

- (1) A formal problem statement connecting ordered JavaScript backtracking traces to NFA accepting-run ambiguity.
- (2) A position-automata construction in Coq for a regular core, including syntax, labeling, nullable/first/last/follow sets, marked-word acceptance, and a basic ambiguity interface.
- (3) A proposed trace/run correspondence theorem that turns ambiguity bounds into bounds on successful backtracking alternatives.
- (4) A ReDoS-oriented interpretation of Weber–Seidl ambiguity classes as semantic certificates for families of hard matching instances.
- (5) A roadmap for completing the mechanized development and extending the regular core toward richer JavaScript features.

2 Motivating Example

Consider the expression

$$(a \mid aa)^*b$$

on inputs of the form $a^n b$ and $a^n c$. A backtracking engine explores ways of decomposing the prefix a^n into repetitions of a and aa . On $a^n b$, some decompositions lead to a successful match; on $a^n c$, the engine may explore many decompositions before discovering that the final literal cannot be matched. This is the familiar shape of a ReDoS example: the engine’s trace tree grows rapidly with n , while the input itself grows linearly.

The automata view says that each decomposition is an accepting path in a nondeterministic automaton for the expression prefix. The engine view says that each decomposition is a branch in an ordered backtracking tree. The same combinatorial object is therefore seen twice: once as an

unordered set of NFA runs, and once as a priority-ordered trace. A proof relating the two views lets us use automata-theoretic ambiguity to reason about the successful frontier of the trace tree.

This example also illustrates a limitation. ReDoS is not only about successful matches. Failing branches can dominate runtime, especially on near-miss inputs. Our bridge is therefore a first layer: it relates successful trace leaves to accepting runs. A complete ReDoS account must also quantify failing prefixes, which we discuss in Section 8.

3 Background

3.1 Backtracking Tree Semantics

Barriere et al. define an inductive semantics for JavaScript regular expressions that produces a backtracking execution trace tree rather than merely the first match [1]. A trace tree contains leaves for success and failure, unary nodes for deterministic operations such as reading a character or opening a capture group, and binary or ordered nodes for backtracking choices. The left-to-right order of choice nodes represents engine priority. The match returned by a conventional backtracking engine is the leftmost successful leaf, while the full tree records every alternative that would be considered by a complete search.

For this paper, we abstract this idea to a regular core. Let $\mathcal{T}(r, w)$ be the ordered trace tree for regular expression r and word w . We write $\text{SuccLeaves}(\mathcal{T}(r, w))$ for the list of successful leaves, preserving priority order. The returned engine match, when it exists, is the head of this list.

3.2 Automata Ambiguity

Let an NFA be $\mathcal{A} = (Q, \Sigma, \delta, I, F)$. A run over $w = a_1 \cdots a_n$ is a path

$$q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} q_n$$

with $q_0 \in I$. It is accepting when $q_n \in F$. The ambiguity of $\mathcal{A}$ on w , written $\text{amb}_{\mathcal{A}}(w)$, is the number of accepting runs of $\mathcal{A}$ on w . The degree of ambiguity of $\mathcal{A}$ is

$$\sup_{w \in \Sigma^*} \text{amb}_{\mathcal{A}}(w).$$

An automaton is unambiguous when this degree is at most one, finitely ambiguous when the supremum is finite, polynomially ambiguous when $\text{amb}_{\mathcal{A}}(w)$ is bounded by a polynomial in $|w|$, and exponentially ambiguous when no polynomial bound exists.

Weber and Seidl give structural criteria and bounds for these classes. In particular, infinite ambiguity is characterized by a simple pattern in the automaton and is decidable in polynomial time; the degree of polynomial ambiguity growth is also computable in polynomial time [2]. These criteria are exactly the kind of finite certificate that a proof assistant can check once the underlying graph theory has been formalized.

4 A Common Representation: Position Automata

We use position automata as the bridge representation. A regular expression is first transformed into a positioned expression in which every atom occurrence is assigned a fresh natural-number position. For example, ab^* becomes $a_1 b_2^*$. The construction computes the usual nullable, firstpos, lastpos, and followpos sets.

The Coq development defines source expressions and positioned expressions as:

$$\begin{aligned} r &::= \emptyset \mid \epsilon \mid a \mid r + r \mid rr \mid r^* \\ \hat{r} &::= \emptyset \mid \epsilon \mid (p, a) \mid \hat{r} + \hat{r} \mid \hat{r}\hat{r} \mid \hat{r}^*. \end{aligned}$$

The labeling function label traverses the source syntax left-to-right and assigns consecutive positions. The automaton states are sets of positions. The start state is $\text{firstpos}(\hat{r})$, the transition from a

state S on symbol a is the union of $\text{followpos}(p)$ for positions $p \in S$ whose label matches a , and finality is determined by intersection with $\text{lastpos}(\hat{r})$, with a nullable special case for the empty word.

The development also defines a marked-word semantics. A marked word records not only the consumed symbols, but the positions that consumed them:

$$[(p_1, a_1), \dots, (p_n, a_n)].$$

Marked words are a useful proof device because they expose the exact syntactic occurrences used by a match. The theorem already mechanized in the development shows that labeling preserves Kleene semantics after erasing positions.

5 Trace/Run Correspondence

The central bridge theorem relates three objects for a regular expression r and input word w :

- (1) successful leaves of the backtracking tree $\mathcal{T}(r, w)$;
- (2) marked matches of the positioned expression $\text{label}(r)$;
- (3) accepting runs of the position automaton built from $\text{label}(r)$.

THEOREM 5.1 (SUCCESSFUL TRACE LEAVES CORRESPOND TO ACCEPTING RUNS). *For every regular expression r in the regular core and every word w , there is a bijection between $\text{SuccLeaves}(\mathcal{T}(r, w))$ and the accepting marked runs of the position automaton for $\text{label}(r)$ over w . Consequently,*

$$|\text{SuccLeaves}(\mathcal{T}(r, w))| = \text{amb}_{\mathcal{A}_{\text{pos}}(r)}(w).$$

The proof decomposes into two parts. First, a successful trace leaf induces a marked word by recording the atom positions read along that branch. The well-formedness of the trace rules implies that consecutive positions respect the followpos relation, so the marked word is accepted by the position automaton. Second, an accepting marked run determines a unique successful branch in the full backtracking tree, because the tree semantics enumerates all alternatives allowed by the syntax. The two translations are inverse up to the irrelevant administrative nodes of the trace tree.

Priority is intentionally not erased completely. The bijection preserves the order induced by backtracking choices. Therefore the first accepting run in trace order corresponds to the match returned by the JavaScript-style engine. Ambiguity counts how many successful alternatives exist; priority chooses one of them.

6 Consequences for Ambiguity

Theorem 5.1 transports standard ambiguity classes to backtracking traces.

COROLLARY 6.1 (UNAMBIGUOUS EXPRESSIONS). *If the position automaton for r is unambiguous, then for every word w the trace tree $\mathcal{T}(r, w)$ has at most one successful leaf.*

COROLLARY 6.2 (FINITE AMBIGUITY). *If the position automaton for r is k -ambiguous, then every trace tree $\mathcal{T}(r, w)$ has at most k successful leaves.*

COROLLARY 6.3 (POLYNOMIAL AMBIGUITY GROWTH). *If the position automaton for r has ambiguity growth bounded by a polynomial p , then*

$$|\text{SuccLeaves}(\mathcal{T}(r, w))| \leq p(|w|)$$

for all w .

These corollaries are deliberately phrased over successful leaves. They do not claim that the full tree is small. A failing input can have many failing branches even when successful ambiguity is low. Nevertheless, successful ambiguity is a semantic invariant that can be checked independently

Table 1. Current Coq development status.

File	Contents	Status
Syntax.v	source and positioned regex syntax; labeling	implemented
Sets.v	finite sets as lists of positions	implemented
KleeneSemantics.v	Kleene and marked semantics	partly proved
PositionAutomaton.v	nullable/first/last/follow and automata	implemented
PositionCorrectness.v	labeling and position lemmas	partly proved
DegreeofAmbiguity.v	NFA ambiguity definitions	implemented
DegreeofInfiniteAmbiguity.v	Weber–Seidl direction	sketch only
LindenBridge.v	bridge to JavaScript regex development	scaffold

of a particular engine implementation, and it provides a foundation for richer analysis of failing prefixes.

7 Mechanization Status

The Coq artifact currently consists of the following components.

The most important completed theorem is that the labeling pass preserves the regular-language semantics after erasing positions. The current automaton file contains a marked acceptance predicate for the Glushkov construction, but the full equivalence between marked regex semantics and marked automaton acceptance is still open. The ambiguity file defines accepting-run counts, k -ambiguity, finite ambiguity, polynomial ambiguity, and a proof that deterministic automata viewed as NFAs are unambiguous.

This division gives a clean proof roadmap:

- (1) complete marked semantics $\Leftrightarrow$ position-automaton acceptance;
- (2) define the regular-core backtracking tree in Coq;
- (3) prove Theorem 5.1;
- (4) formalize Weber–Seidl graph patterns as checkable certificates;
- (5) connect those certificates to ReDoS-oriented warnings.

8 Implications for ReDoS

ReDoS arises when a regular-expression engine spends superlinear or exponential time exploring alternatives. The bridge developed here suggests a semantic pipeline for certified ReDoS analysis.

Step 1: Compile to a position automaton. For the regular core, this is syntax-directed and already implemented. For larger JavaScript fragments, compilation must account for captures and zero-width assertions, or else restrict the analysis to a conservative regular approximation.

Step 2: Classify ambiguity. Weber–Seidl criteria can identify unambiguous, finitely ambiguous, polynomially ambiguous, and exponentially ambiguous automata. In a verifier, the analyzer can emit a graph certificate and Coq can check that the certificate implies the claimed ambiguity class.

Step 3: Interpret the class as a trace bound. The trace/run theorem turns ambiguity bounds into bounds on successful alternatives. For many practical patterns, these alternatives are the source of the engine’s branching behavior.

Step 4: Account for failure. A full ReDoS detector must also bound failing branches. We propose to model failing branches through *prefix ambiguity*: the number of partial runs that consume a

prefix and reach a live but ultimately rejecting configuration. This quantity is naturally expressible as ambiguity in product automata that track both progress and residual obligations. Formalizing this prefix layer is the main technical extension beyond the successful-leaf correspondence.

9 Related Work

JavaScript regex semantics. Barriere et al. provide the closest semantic foundation: a complete and mechanized JavaScript regex semantics that produces ordered backtracking trees and is proved faithful to the ECMAScript specification via Warblre equivalence [1]. Our work is complementary. Rather than verifying an engine algorithm such as PikeVM, we use their tree perspective as the engine-side semantic object to be related to automata ambiguity.

Automata ambiguity. Weber and Seidl’s work is the classical source for structural analysis of NFA ambiguity [2]. Their criteria for infinite ambiguity and polynomial ambiguity growth are especially relevant because they turn a semantic growth question into a finite graph problem. Our contribution is to position these criteria as certificates for backtracking trace growth.

ReDoS analysis. Existing ReDoS tools typically search for syntactic vulnerability patterns or construct witness strings for catastrophic backtracking. The mechanism proposed here aims at a proof-producing alternative: a detector should justify its warning by an automata ambiguity certificate and a verified connection to the engine semantics. The approach is not yet a complete detector, but it clarifies which semantic property a detector should certify.

10 Conclusion

Backtracking traces and automata ambiguity describe the same source of nondeterminism from different angles. Traces preserve the priority discipline of JavaScript-style engines; ambiguity abstracts away priority and counts the number of successful decompositions. By proving that successful trace leaves correspond to accepting runs of a position automaton, we can transport decades of automata-theoretic ambiguity results into the analysis of backtracking regular-expression execution.

The current Coq development establishes the regular-expression and position-automata infrastructure and the basic ambiguity interface. The next milestone is the mechanized trace/run correspondence theorem. Once completed, the development will provide a foundation for proof-producing ReDoS analyses that connect engine behavior, automata structure, and verified certificates.

References

- [1] Aurèle Barrière, Victor Deng, and Clément Pit-Claudel. 2026. Formal Verification for JavaScript Regular Expressions: A Proven Mechanized Semantics and Its Applications. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 68 (2026), 30 pages. doi:10.1145/3776710
- [2] Andreas Weber and Helmut Seidl. 1991. On the Degree of Ambiguity of Finite Automata. *Theoretical Computer Science* 88, 2 (1991), 325–349. doi:10.1016/0304-3975(91)90381-B